# Experiment - 8

**Student Name:** Reyansh Goswami
**UID:** 23BCS11136
**Branch:** BE-CSE
**Section/Group:** KRG_2B
**Semester:** 6th
**Date of Performance:** 08/04/26
**Subject Name:** System Design
**Subject Code:** 23CSH-314

## Aim:

To design and analyze a scalable online movie ticket booking system that allows users to browse movies, select seats, and make payments in real time, ensuring high availability, concurrency handling, and low latency.

## Objectives:

1. To understand system design principles for large-scale booking systems.
2. To design APIs for movie listing, seat booking, and payment services.
3. To identify functional and non-functional requirements.
4. To handle concurrency issues during seat booking.
5. To design a high-availability and scalable system architecture.

## Procedure-

1. Identify functional requirements of the movie ticket booking system.
2. Define non-functional requirements such as scalability, availability, and latency.
3. Analyze concurrency challenges in seat booking.
4. Identify core entities such as User, Movie, Theatre, Show, Seat, and Booking.
5. Design the system architecture using Draw.io.
6. Validate the design for booking, payment, and seat allocation scenarios.

## Functional Requirements –

1. Users should be able to do the registration, login and KYC verification.
2. Application should allow users to view real-time stocks prices & its historical data.
3. Application should support placing, modifying, and cancelling orders along with limiting orders with accurate status updates.
4. Application should provide users with a watchlist with real-time updates.

5. Application should provie a dashboard to track current holdings, trade history, P & L, and performance insights.

## Non-functional Requirements-

1. Scale: 2 - 3M DAU with high-frequency trades, along with 8 - 10k stocks.
2. CAP Theorem:
   Consistency >>> Availability
   Correctness of stocks is more important than uptime.
   (A wrong balance, duplicate trade, or stale price can lead to huge financial losses)
   Getting the stocks prices in real-time (highliy available)
3. Latency:
   Order placement < 100ms, market data (updated stock value) < 50ms

## Core Entities:

1. Users
2. Stocks
3. Orders
4. Trade
5. Portfolio
6. Watchlist

## Api Design:

**Users:**

1. POST: / auth / signup
2. POST: / auth / verify-KYC
3. POST: / auth / login

**Market Data:**

1. WS: / api / v1 / stocks [Listing of all the stocks]
2. GET: / api / v1 / stocks / (stock_symbol_id) [stock details]
3. GET: / api / v1 / stocks / {stock_symbol_id} / history [Historical Data]

**Funds:**

1. POST: / api / v1 / funds / deposit
2. POST: / api / v1 / funds / withdraw
3. GET: / api / v1 / funds / history

**Trading:**

1. POST: / api / v1 / orders [Place buy / sell order]
2. GET: / api / v1 / {user_id} / orders [List user orders]
3. GET: / api / v1 / orders / {order_id} [Order Status]
4. DELETE: / api / v1 / orders / [order_id] [Cancel order]

**Portfolio:**

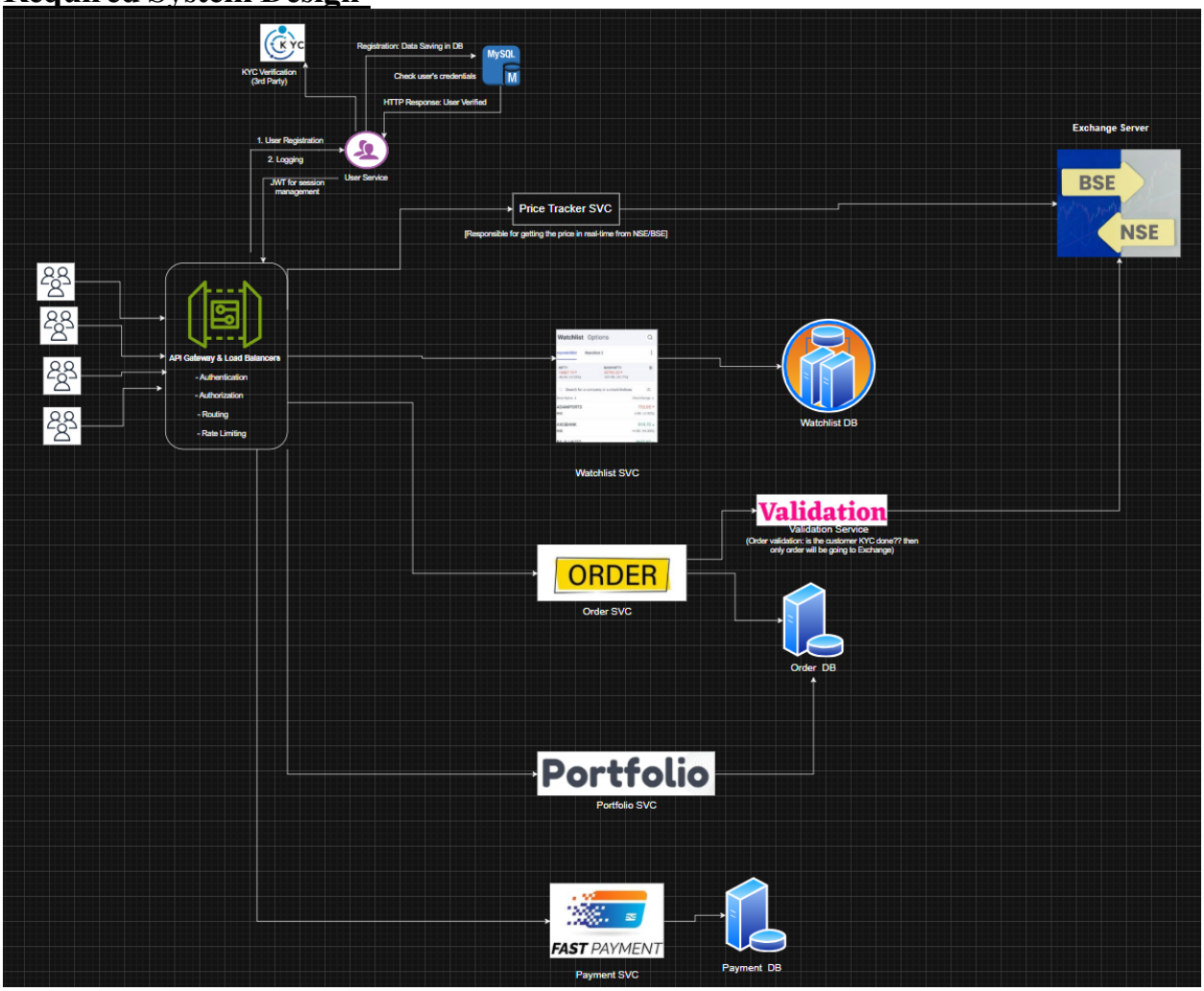
1. GET: / api / v1 / portfolio
2. GET: / api / v1 / portfolio / holdings
3. GET: / api / v1 / portfolio / positions
4. GET: / api . v1 / portfolio / performance

[User_id will be sent in header not in request to secure the request]

**Watchlist:**

1. GET: / api / v1 / watchlists
2. POST / api / v1 / watchlists / {watchlist_id} / symbols

## Required System Design-




## Outcome / Result -

The online movie ticket booking system was successfully designed with features like movie browsing, seat selection, secure payments, and concurrency handling, ensuring scalability, high availability, and efficient real-time booking under heavy load conditions.